Ruben Doste, Blanca Rodriguez

PRACTICAL 1:

Single cell modelling and simulation

COMPUTATIONAL MEDICINE

DEPARTMENT OF COMPUTER SCIENCE, UNIVERSITY OF OXFORD | HILARY TERM

1. Introduction to MATLAB

In this practical session, we will be utilising the Hodgkin and Huxley models and the human ventricular cellular model (ToR-ORd), introduced in the lectures 7 and 8. We will be using MATLAB, widely recognized as a popular coding language in computational modelling and simulation, primarily owing to its simplicity, widespread use and number of developed algorithms and applications. In addition to its prevalence, MATLAB is well-suited for prototyping, allowing researchers and students to quickly develop and test computational models. The commands employed during this practical will be simple, focusing on fundamental aspects, such as basic plotting, utilization of the colon operator, and working with data structures. This approach ensures accessibility for participants, even those with limited MATLAB experience. For further guidance and assistance in navigating this practical, relevant documentation can be found at:

https://eng.ox.ac.uk/media/8038/matlab_notes.pdf

2. The Hodgkin and Huxley model

The Hodgkin and Huxley model (HH, *J Physiol 1952*) is a mathematical description of the action potential (AP) in neurons. It consists of a set of nonlinear differential equations, which have to be solved with numerical analysis techniques.

The cell membrane is represented as a capacitance (**C_m**), with three different ionic currents between the intra- and extra-cellular spaces:

- Na^+ current (**I_{Na}**): Depolarisation
- K^+ current (**I_K**): Repolarisation
- Leak current (**I_{leak}**): Every other current

Each membrane current (**I_X**) is computed as the product of the ionic conductance (**g_X**) and the driving force, i.e. the difference between the actual membrane potential (**V**) and the equilibrium potential of the specific ion (**V_X**). Some current conductances depend on voltage and time, and this dependence is accounted for by one or more gating variables, which represent the opening/closing of the channels, and always vary between 0 and 1.

$$C \cdot \frac{dV}{dt} = I_C$$

$$I_C = I_{stim} - (I_{Na} + I_K + I_{leak})$$

$$I_{Na} = g_{Na} \cdot m^3 \cdot h \cdot (V - V_{Na})$$

$$I_K = g_K \cdot n^4 \cdot (V - V_K)$$

$$I_{leak} = g_{leak} \cdot (V - V_{leak})$$

The total current (**I_C**) consists of the sum of all the ionic currents, and the external stimulus current (**I_{Stim}**).

The Hodgkin and Huxley model has **4 state variables**:

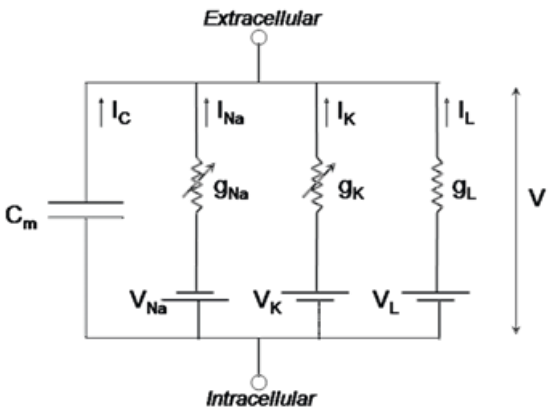
- **V**: membrane voltage
- **m**: I_{Na} activation gate
- **n**: I_{Na} inactivation gate
- **h**: I_K activation gate

$$\frac{dm}{dt} = \alpha_m(V) \cdot (1 - m) - \beta_m(V) \cdot m$$

$$\frac{dh}{dt} = \alpha_h(V) \cdot (1 - h) - \beta_h(V) \cdot h$$

$$\frac{dn}{dt} = \alpha_n(V) \cdot (1 - n) - \beta_n(V) \cdot n$$

When stimulated, the neuron fires a train of APs. The shape and frequency of these APs depend on the type of stimulus. Additional information regarding the Hodgkin-Huxley model can be found here.



The H&H model has been implemented in MATLAB in the function file `modHH`, which takes as input the time (`t`) and the state variable vector (X) and gives as output the derivative of the state variables in time (dX). The function is called by the `MainFile_modHH.m` which solves the model by using the built-in function `ode15s` and gives as outputs a series of figures, showing the stimulus current and the corresponding AP.

○ *Test the Hodgkin and Huxley Model*

By using the MATLAB files provided, simulate the action potential using the HH neuron model, and have a look at the outputs (as shown in the introductory slides):

- Open MATLAB (Linux terminal→ `matlab`) and navigate using MATLAB interface to the folder with the files: *`MainFile_modHH.m`* and *`modHH.m`*

- Run *`MainFile_modHH.m`* and explore the results.
- Have a look at the script and the function file, containing the HH model. Try to understand how these file works by reading the comments and based on the model description given above.
- The code includes some lines that are commented (with a single %): you can uncomment them to generate additional figures, i.e. all the ionic currents (Figure 2) and all the state variables (Figure 3).

Changes of the stimulus amplitude

- By default, the model is stimulated with an impulse current of 80 uA/uF for 0.5 ms, with a cycle length (CL) of 100 ms. This means that the neuron fires every 100 ms (10 times per second): you can see one AP is elicited after each external stimulus is applied.
- Modify the main file to change the magnitude of the stimulus current, run the file again, and try to understand how the AP shape and frequency change. What happens if the stimulus current is too low or too high? Do you notice any change in the neuron behaviour?

Changes of the stimulus frequency

- With the default cycle length (CL) of 100 ms, the neuron fires every 100 ms (10 times per second), just after each external stimulus is applied. The CL defines the distance between one stimulus and the next one.
- Modify the main file to change the frequency of the stimulus current and look at how the AP shape and frequency change. What happens when the CL is too small?

Euler's Method

- The script `Euler_HH.m` also solves the HH equations using Euler's method and compares the results with those obtained by the `ODE15s` solver.
  Complete the implementation of the Euler's method by filling in the missing lines **29**,**30**,**31** and **41**. Solve the equations for 0.5 s with a time step of 0.05 ms. Please, note that the functions α_m, α_n, α_h and β_m, β_n, β_h are defined at the end of the script.
  After obtaining the results, compare the resulting voltage and gating variables with those provided by the ODE15s solver. What is causing the differences between both methods? How can they be reduced?

3. ToR-ORd model

The ToR-ORd model is the most accurate electrophysiological model for the electrical activity of the ventricular cardiomyocytes. This model represents one of the latest advancements in ventricular models developed to date. Throughout the session, students will acquire hands-on experience working with the ToR-ORd model, enabling them to explore various model properties and comprehend its applications in the study of cardiac safety and arrhythmogenicity. ToR-ORd model structure is similar to the neuronal model developed by Hodgkin & Huxley in 1952, but including more cardiac specific channels and pumps, therefore more state variables and a more complex intracellular compartmentalisation.

a. Research article

A description of its construction, calibration and validation can be found in the original open-source publication, in the scientific journal eLife (https://doi.org/10.7554/eLife.48890). Please, download the pdf document and read it briefly. Most of the equations used in the model are an update of a previous model, the Rudy O'Hara model (ORd) (https://doi.org/10.1371/journal.pcbi.1002061). All the model equations can be found in the supplementary material.

b. Download

Download the MATLAB code following the GitHub link at the end of the methods section and copy the folder in your working directory.

4. Run example script

a. Code logic

Information about the code structure is available in the documentation, specifically in the "User Guide" file. Pay special attention to Section 2, where the various layers of the code are explained.

- Open the function `model_Torord` and explore how the different equations described in the previous papers are implemented. How many state variables are used? Can you identify the ionic currents used to update the membrane voltage? What ionic current is implemented using a Markov model?

b. Run example scripts 1 2 3

To familiarise yourself with the script structure, run the three first example scripts provided with the code. Please note the following:

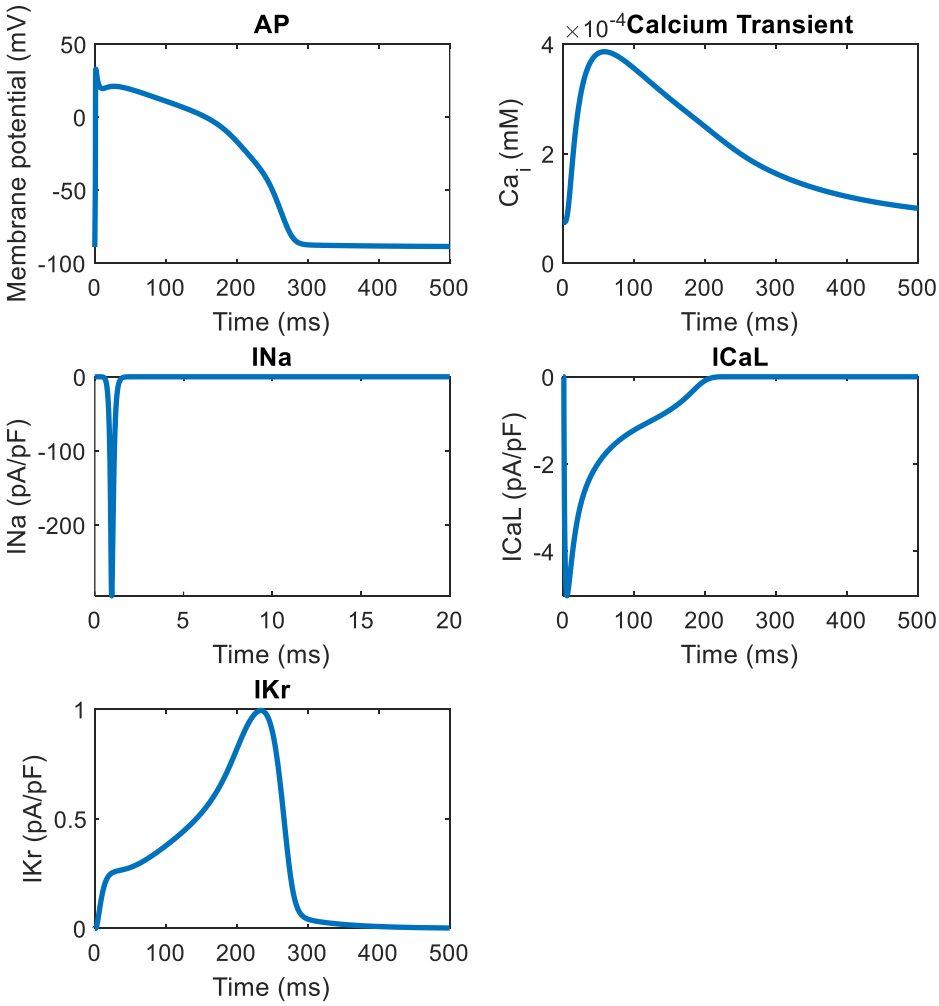
- Model parameters are adjusted within the "params" structure. Some of the parameters used in the simulations include the basic cycle length, the model function, the number of simulated beats, channel multipliers, or the cell type.
- Equations are solved using the `ode15s` function, which is the solver employed in MATLAB for solving ordinary differential equations (ODEs).
- By utilising the colon operator and parallelisation `parfor`, different models or conditions can be solved concurrently. This is clearly shown in script 2, where different multipliers for current IKr are simulated.

Explore the previous scripts by changing the parameters or plotting different currents. Create new scripts to avoid modifying the originals.

5. Single cell simulations

a. *Prepare a script to visualize currents (INa, ICaL, IKr)*

Modify the Script 1 to plot, in the same figure, the action potential (AP), Calcium transient (Cai), Sodium Current (INa), Calcium current (ICaL) and the rapid delayed rectifier potassium current (IKr). Use the MATLAB function `subplot` with the input `subplot(2,3,i)` and obtain an image similar to the one provided below. Can you explain the difference between Calcium transient and ICaL current?



b. *Rate Dependecy*

Observe and plot V, Ca^{2+}_i, INa, ICaL and IKr for the default basic cycle lengths (BCLs) of 1000 and using 300 ms. Plot the variables after simulating 200 beats.

Hint: Use a copy of sample Script 2. This time, you need 2 different `param` structure (each with a different BCL field). Remove lines 10 to 21, and initialise the array of parameters (`params`) as:

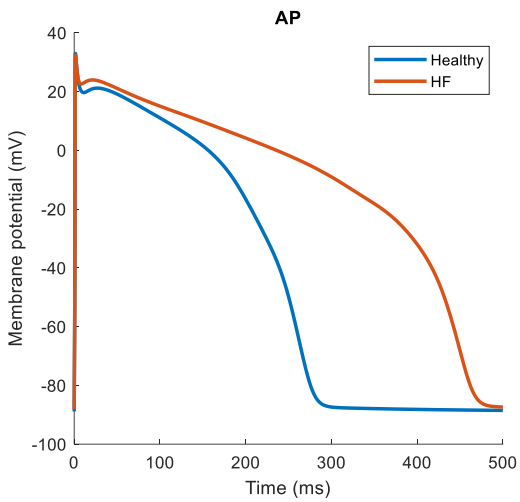
```
params(1:2) = param;
```

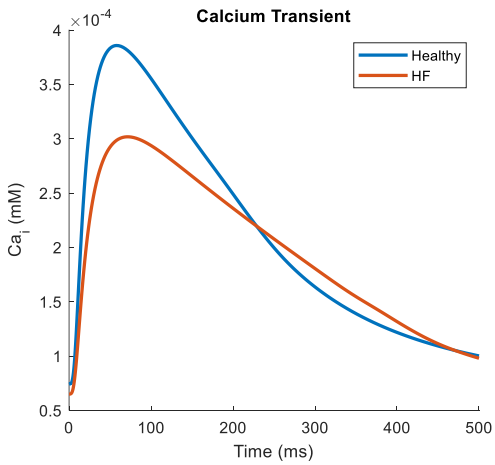
Describe which phenomena are observed.

6. Heart Failure modelling

Build a human AP model of Heart Failure, by scaling some parameters of the ToR-ORd model based on the Table here below. Create a new script where all these changes are defined before running the model. All these parameters can be modified by including each current multiplier in the `params` structure, as in the sample Script 2. The list of parameters could also be extracted by having a look at the "`model_Runner`" function. Compare the new AP and calcium transients with the healthy ones.

Parameter	Scaling Factor
GNaL	1.5
GNaCa	1.7
PNaK	0.5
GKr	0.7





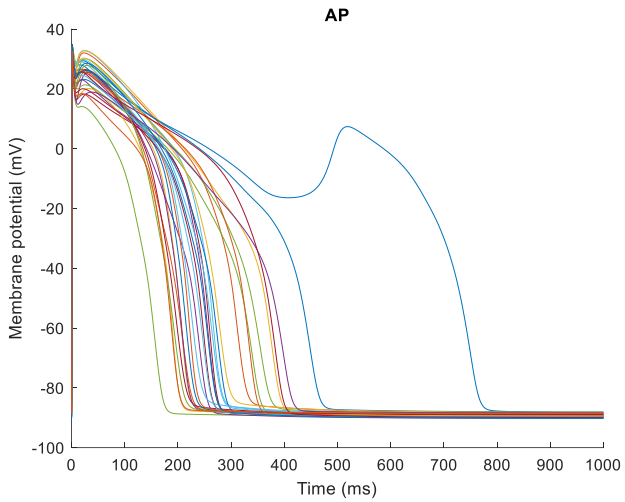
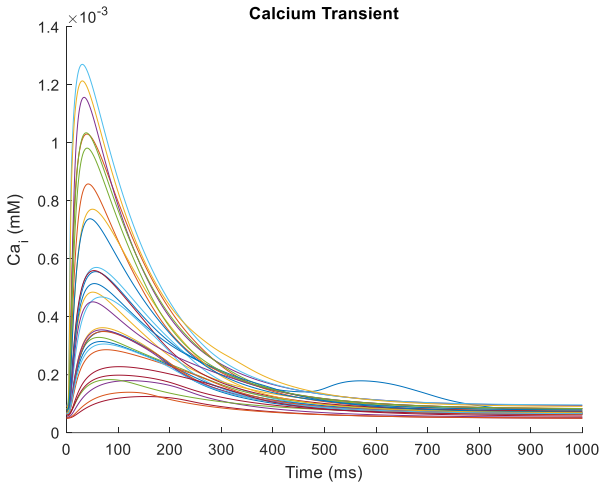
7. Population of models

Following a similar approach as in the previous section, a population of models can be created by generating parameters for the ionic channels using the Latin hypercube sampling algorithm. We are going to generate a population of 30 models by changing 7 different parameters, corresponding to the following ionic currents: ICaL, INaL, INaCa, INaK, IKr, IKs, Ito.

To achieve this:

- Create a matrix "LHS" using the MATLAB function "`lhs design`" that contains the sampled values for the 30 models and 7 parameters.
- Rescale all these values between 0.5 and 2 in a matrix called "LHSR" using the function "`Rescale`"
- Each of the 30 models of the generated population will be defined by the parameters located in each row of the LHS matrix. Now, you need to assign the generated parameters to each current multiplier, taking into account the column number, as it is done in the row 20 of the demo Script 2.

  As an example:

```
params(iParam).ICaL_Multiplier = LHSR(iParam,1);
```

Run the simulations for a BCL of 1000ms and simulate 20 beats per model. Plot the resulting APs and CaTs of the population of models.